

Ablation

Component	1×		3×		10×	
	with	without	with	without	with	without
Full system , macro exact match (%)	78.3		81.7		85.0	
model calls per item (mean / cap)	1.0 / 1		3.0 / 3		4.4 / 10	
how a misread line gets fixed	code repairs only		outvoted by two other reads		code flags it, model re-reads it	
One read of the notes						
Code solver (<i>vs Granite answering directly</i>)	78.3	0.0	81.7	–	85.0	–
Worked examples in the prompt	78.3	48.3	81.7	51.7	85.0	66.7
Restate each line in plain words first	78.3	63.3	81.7	68.3	85.0	70.0
Code repair (no model calls)						
Wrong-name repair	78.3	70.0	81.7	76.7	85.0	80.0
Extra calls, 3×: read three times and vote						
2nd and 3rd read + per-line vote (<i>vs one read</i>)	–	–	81.7	78.3	85.0	86.7
Reads use different prompts (<i>vs one prompt</i> × 3)	–	–	81.7	80.0	85.0	85.0
Extra calls, 10×: re-ask only the lines code flags						
All targeted re-asks (<i>vs the 3-read vote alone</i>)	–	–	–	–	85.0	81.7
<i>each kind of re-ask removed on its own:</i>						
name check – <i>value not in its line</i>	–	–	–	–	85.0	86.7
missed line – <i>“no fact” but names people</i>	–	–	–	–	85.0	80.0
order – <i>who comes first</i>	–	–	–	–	85.0	85.0
line type – <i>kind of statement</i>	–	–	–	–	85.0	85.0
contradiction – <i>lines cannot both hold</i>	–	–	–	–	85.0	85.0

Table 1: The ablation. Macro exact match (%), mean of 1 runs on 60 items. Each row removes one component from the system at that budget and keeps everything else; “with” is always the full system at that budget. –: the component does not exist at that budget. “2nd and 3rd read” at 3× without = the 1× system.

How to read it. Read across a row to see what one component is worth at each budget: the gap between *with* and *without*. Read down the shaded rows for the cost–accuracy curve. The budgets are nested: 3× is the 1× system plus two more reads, and 10× is the 3× system plus targeted re-asks, so every 1× component is also inside 3× and 10×, and every 3× component inside 10×. With 60 items, one item moves the macro score by 1.7 points; differences smaller than the run-to-run spread ($\pm$) are noise.

How the extra calls fix errors, and why 3× and 10× do it differently. At 1× a misread line can only be caught by code: the wrong-name repair swaps or drops a fact that names someone its line never mentions. Nothing can recover a line the model missed. At 3× the system never asks the model to correct itself. It reads the whole note three times, each with a differently formatted prompt, and keeps, line by line, the fact at least two reads agree on. This fixes *random* misreads; it cannot fix a mistake all three reads share, which is why the reads use different prompts. At 10× code decides *which* lines need another look and *why*. After the vote it flags lines that break a check or on which the reads disagreed, groups them by kind of problem, and makes one call per kind, showing the model only those lines, the earlier readings, and what is wrong. A re-read replaces the vote only when code had found a concrete problem and the new answer passes the checks; otherwise it counts as one more vote. So 3× fixes errors by *agreement*, and 10× by *targeted correction*.

1 runs per system, 60 items. Command: `python3 ablate.py --suite full --runs 1 --workers 16`. Started 2026-10-01 23:19, took 21.3 min.

Supporting tables

Prompt techniques at 1×

	Unique	Ambiguous	Inconsistent	Macro	Calls/item	Lines right	Missed	Wrong
★ Few-shot: 12 worked examples, restate each line	80.0	75.0	80.0	78.3	1.0	94.5	46	17
One-shot: 1 worked example	50.0	45.0	65.0	53.3	1.0	91.6	67	28
Zero-shot: rules and hints only	50.0	40.0	55.0	48.3	1.0	90.3	76	34
No plain-words restatement	65.0	60.0	65.0	63.3	1.0	94.1	55	12
Plain field names (<i>a/b</i> instead of <i>earlier/later</i>)	80.0	70.0	80.0	76.7	1.0	94.7	37	23
Granite answers the whole item directly (no solver)	0.0	0.0	0.0	0.0	1.0	–	–	–

Table 2: Each row is one 1× system that differs from the submitted one (★) only in its prompt, all scored with the wrong-name repair. *Lines right*: % of note lines whose facts match the hand-checked facts exactly; *Missed*: real constraints read as “no fact”; *Wrong*: a wrong fact, or a fact invented from filler (counts per run, over all items). Worked examples use made-up names but were written from errors seen on the visible notes, so they are the main thing to watch for overfitting.

Code repairs at 1× (same model output)

	Unique	Ambiguous	Inconsistent	Macro	Calls/item	Lines right	Missed	Wrong
None	70.0	75.0	65.0	70.0	1.0	93.7	45	27
★ Wrong-name repair	80.0	75.0	80.0	78.3	1.0	94.5	46	17
Station/time swap repair	70.0	75.0	65.0	70.0	1.0	93.7	45	27
Both	80.0	75.0	80.0	78.3	1.0	94.5	46	17

Table 3: All four rows score the *same* 1× model output, so differences come from the repair alone, not sampling. Wrong-name repair: a fact naming someone its own line never mentions is swapped to the one unused name the line does mention, else dropped. Swap repair: a station written where a time belongs (or the reverse) becomes the matching fact; it fires too rarely to matter and is not in the submitted system.

Ways to spend three calls

	Unique	Ambiguous	Inconsistent	Macro	Calls/item	Lines right	Missed	Wrong
1 read (the 1× system)	80.0	75.0	80.0	78.3	1.0	94.5	46	17
2 reads + 1 re-ask of the lines they disagree on	90.0	80.0	85.0	85.0	2.7	96.0	32	14
3 reads, same prompt, per-line vote	85.0	70.0	85.0	80.0	3.0	95.2	38	17
★ 3 reads, 3 different prompts, per-line vote	85.0	75.0	85.0	81.7	3.0	95.5	41	10

Table 4: Same prompt and repair throughout; only the use of the extra calls changes. A re-ask lets one fresh sample overwrite a line, so it can turn a right reading wrong; a vote needs two reads to agree. Reads with the same prompt make the same mistakes, so they rarely outvote one another.

Where the 10× calls go

	Items asked	Lines re-asked	Macro without it	Gain
3-read vote (the 3× system)	–	–	81.7	–
+ name check (<i>a name, time or station the line does not mention</i>)	40%	36	86.7	-1.7
+ missed line (<i>read as no fact, but the restatement names people</i>)	45%	39	80.0	+5.0
+ order (<i>reads disagree on who comes first</i>)	20%	13	85.0	+0.0
+ line type (<i>reads disagree on the kind of statement</i>)	37%	38	85.0	+0.0
+ contradiction (<i>two lines cannot both hold</i>)	2%	3	85.0	+0.0
★ Full 10×	–	–	85.0	–

Table 5: After the 3-read vote, code flags lines and makes one call per kind of problem, only for items that have such lines. *Items asked*: share of items that got this call; *Lines re-asked*: lines sent, per run; *Gain*: full 10× minus the system without this kind of call.

Declared versus true kind

True kind $\rightarrow$	1 $\times$ U			A			I		
	1 $\times$ U	A	I	3 $\times$ U	A	I	10 $\times$ U	A	I
Declared unique	16	0	1	17	0	1	18	0	1
Declared ambiguous	4	18	1	3	19	1	1	19	0
Declared inconsistent	0	2	18	0	1	18	1	1	19

Table 6: Mean item counts over runs for the submitted system at each budget (U = unique, A = ambiguous, I = inconsistent); the diagonal is correct. A missed constraint leaves too many solutions (a unique item declared ambiguous); a misread one usually makes the notes contradict (declared inconsistent).